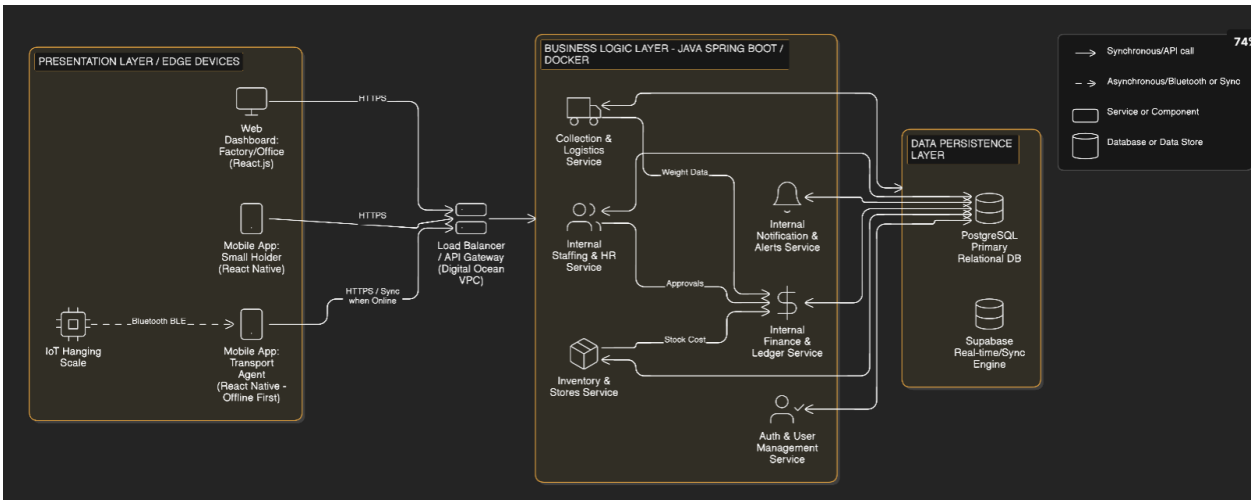
System Architecture Document (SAD): Project Dalupotha

Version: 1.0



1. Architectural Overview

The Dalupotha System is designed using a self-contained **Microservices Architecture**. This approach isolates core business domains (e.g., field collection, financial ledgers, inventory, and human resources) into independent, highly cohesive services. This allows the factory to scale specific modules—such as the collection service during peak harvesting seasons—without impacting the overall system stability.

Layered Architecture:

- **Presentation Layer (Web/Mobile):** The user-facing interfaces. It utilizes responsive web portals for factory management and cross-platform mobile applications for field operators (Transport Agents) and Small Holders.
- **Business Logic Layer:** A cluster of containerized microservices handling the core rules, calculating leaf deductions, managing internal staff roles, processing internal financial ledgers, and routing approval workflows.
- **Database Layer:** A hybrid data tier utilizing traditional relational databases for transactional integrity (like balance payments) and real-time/offline-sync capable databases for remote mobile field operations.

2. System Components

The system is divided into modular, fully independent components to ensure comprehensive operational coverage:

- **Authentication & User Management Module:** Handles all internal user mapping, role management (RBAC), and secure credential issuance for Small Holders, generating JWTs for session management.
- **Collection & Logistics Module:** Manages the integration with Bluetooth IoT scales, records GPS coordinates, and handles the Transport Agent's field operations and offline data caching.
- **Internal Financial & Payout Module:** A standalone financial engine that calculates balance payments, applies quality deductions, and tracks advances and debts on internal ledgers.
- **Inventory & Stores Module:** Tracks stock levels for leaf bags and fertilizers locally, processing requests from Small Holders and routing them to the Store Keeper.
- **Internal HR & Staffing Module:** Manages the factory hierarchy, tracking shifts, approvals, and permissions for internal staff (Factory Staff, Extension Officers, Managers) without external dependencies.
- **Notification & Alerts Service:** An internal messaging queue responsible for pushing in-app notifications, dashboard alerts, and generating internal codes for Small Holder verification and TRI Circular distribution.

3. Technology Stack

- **Frontend (Web):** React.js (for high-performance management dashboards).
- **Frontend (Mobile):** React Native Expo (allows a unified codebase for Android/iOS, crucial for Transport Agents and Small Holders).
- **Backend:** Java Spring Boot (provides robust, enterprise-grade microservices for financial and logistical processing).
- **Database:** PostgreSQL (primary relational database for transactional consistency) and Supabase (utilized specifically for its robust offline-sync capabilities and real-time WebSockets to support remote Transport Agents).
- **Cloud Infrastructure & Hosting:** Docker containers deployed on Digital Ocean Droplets (or managed Kubernetes) for isolated, scalable service execution within a self-managed environment.

4. Database Design

The database follows a normalized relational model to prevent data anomalies and securely house all financial and personnel data internally.

Core Tables:

- Users: UserID, Role (MG, EXT, TA, SK, FT, ST), Contact, HashedPassword, Status.
- SmallHolders: SupplierID, UserID, PassbookNo, LandName, GPS_Lat, GPS_Long, InChargeNo.
- LeafCollections: CollectionID, SupplierID, TransportAgentID, GrossWeight, NetWeight, GPSLocation, Timestamp, SyncStatus.
- FinancialLedger: TransactionID, SupplierID, TransactionType (Advance, Debt, Payout), Amount, Date, Status.
- Inventory: ItemID, ItemType (Fertilizer, Bag), Quantity, LastUpdated.
- ServiceRequests: RequestID, SupplierID, RequestType (MachineRent, Advisory), Status.

5. Deployment Architecture

To ensure high availability, the deployment architecture isolates the database from application nodes and balances incoming traffic entirely within the factory's managed cloud ecosystem.

- **Server Configuration:** Stateless Docker containers running the Java Spring backend, allowing horizontal scaling.
- **Cloud Structure:** Deployed within a Virtual Private Cloud (VPC) to restrict public internet access to the database layer. Application Programming Interfaces (APIs) manage traffic from mobile apps securely.
- **Load Balancer:** A Load Balancer routes incoming web and mobile traffic across available backend containers, ensuring no single point of failure.
- **Backup Server:** Automated daily snapshots of the PostgreSQL volume and continuous WAL (Write-Ahead Logging) archiving to secure Object Storage for disaster recovery.

6. Security Architecture

- **Authentication Method:** OAuth 2.0 with JWT (JSON Web Tokens) for stateless, secure API communication across all Web and Mobile clients. System-generated verification codes handle Small Holder access.

- **Password/Credential Encryption:** System passwords and critical authentication secrets are hashed using bcrypt with adaptive work factors.
- **Role Management:** Granular RBAC applied at the API Gateway and service level. For instance, a token containing the TA (Transport Agent) role cannot successfully POST to the /api/inventory/approve endpoint.
- **Data Encryption:** * *In Transit:* All communications between the mobile app, web portal, and backend are encrypted using TLS 1.3 (HTTPS).
    - *At Rest:* Sensitive data, such as Passbook Numbers and internal financial ledgers, are encrypted at the database level using AES-256.